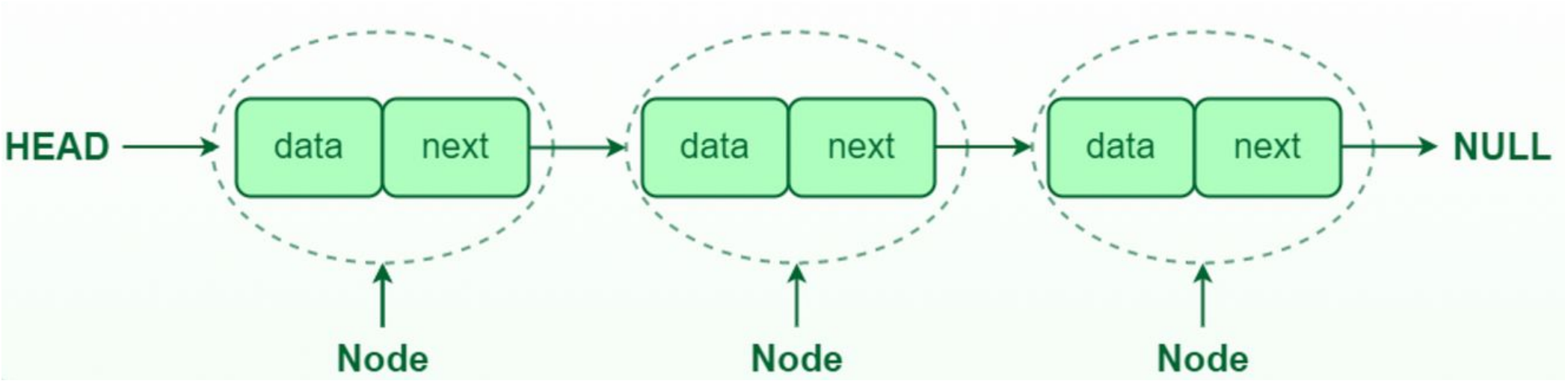
Introduction to Algorithms

Levon Prazyan

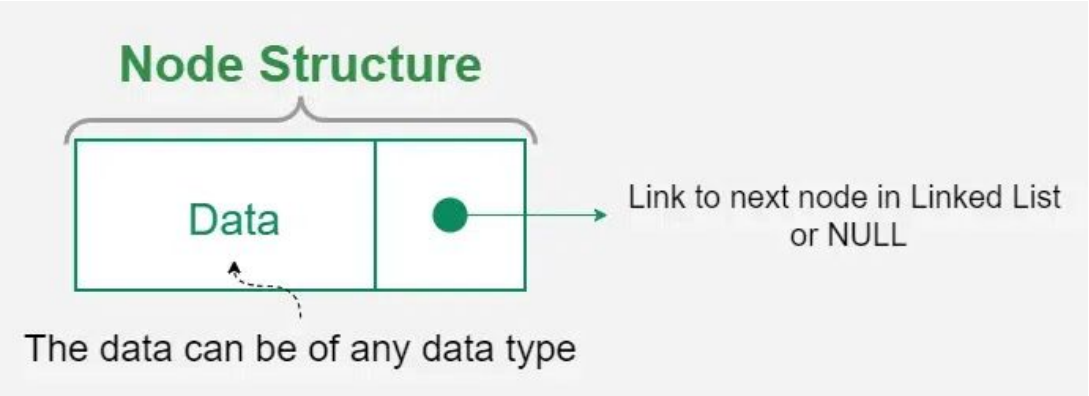
Course Overview

- **Introduction**
- **Sorting and Searching**
- **Arrays and Lists**
- **Stacks and Queues, Double Ended Queue**
- **Heaps**
- **Trees**
- **Hash Tables**
- **Dynamic Programming**
- **Greedy Algorithms**
- **Graph Algorithms**
- **Network Flow**
- **NP-Completeness**

Single-Linked List



Singe-Linked List. Node class



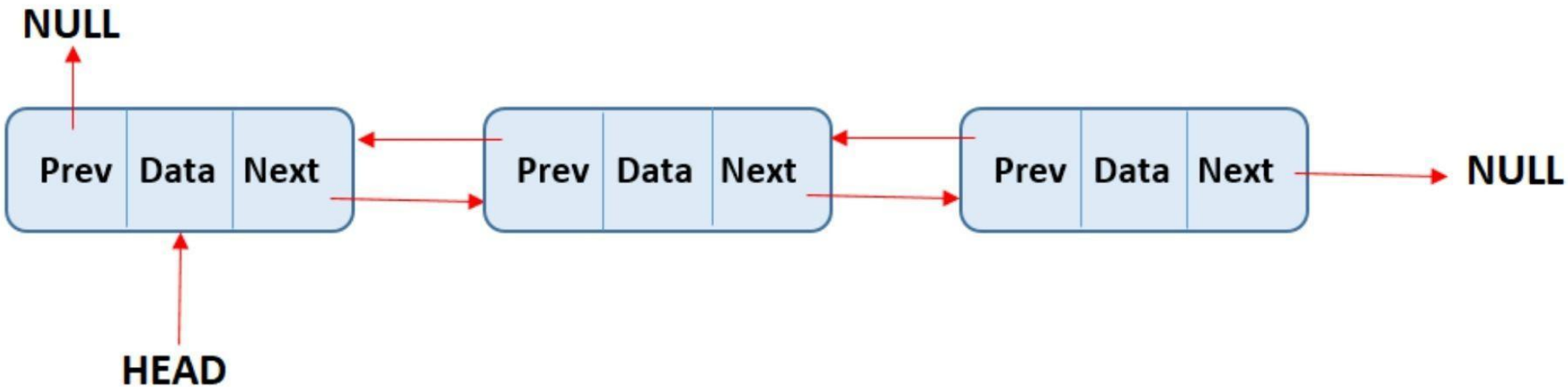

```
class Node
{
public:
T data;
Node* next;

Node(T value)
    : data(std::move(value))
    , next(nullptr)
    {}
};
```

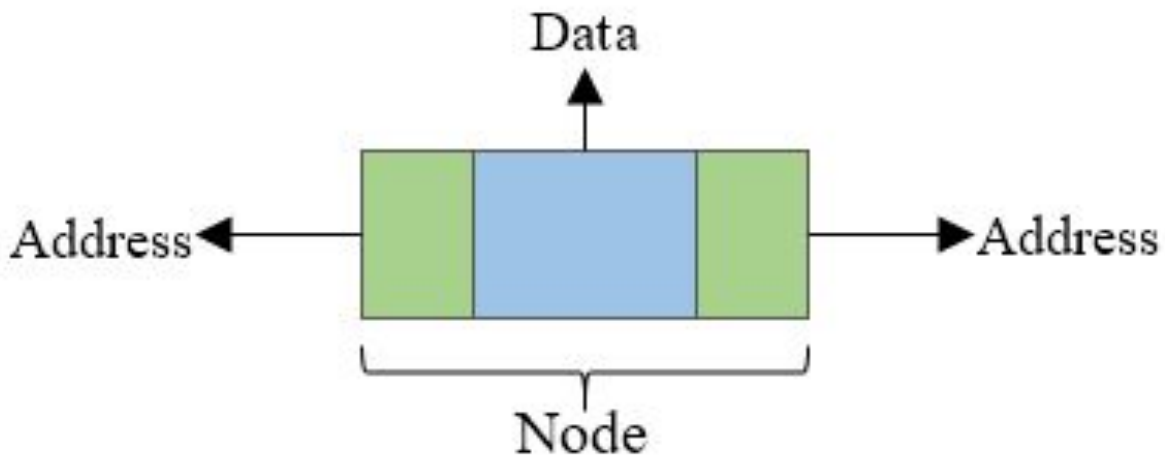
Singe-Linked List class

```
template <typename T>
class SingleLinkedList
{
public:
    bool empty() const;
    std::size_t size() const;
    Node* first();
    Node* last();
    template<typename... Ts> void insert(Node* p_node, Ts&&... values);
    template<typename... Ts> void push_front(Ts&&... values);
    template<typename... Ts> void push_back(Ts&&... values);
    void remove(const T& value);
    void clear();
    void splice(SingleLinkedList<T>& other);
    void reverse();
private:
    Node* m_head;
};
```

Double-Linked List



Double-Linked List Node




```
class Node
{
public:
T data;
Node* prev;
Node* next;

Node(T value)
    : data(std::move(value))
    , prev(nullptr)
    , next(nullptr)
    {}
};
```

Double-Linked List class

```
template <typename T>
class DoubleLinkedList
{
public:
   bool empty() const
   std::size_t size() const
   Node* first()
   Node* last()
   template<typename... Ts> void insert(Node* p_node, Ts&&... values)
   template<typename... Ts> void push_front(Ts&&... values)
   template<typename... Ts> void push_back(Ts&&... values)
   void remove(const T& value)
   void clear()
   void splice(SingleLinkedList<T>& other)
   void reverse()
private:
   Node* m_head;
};
```

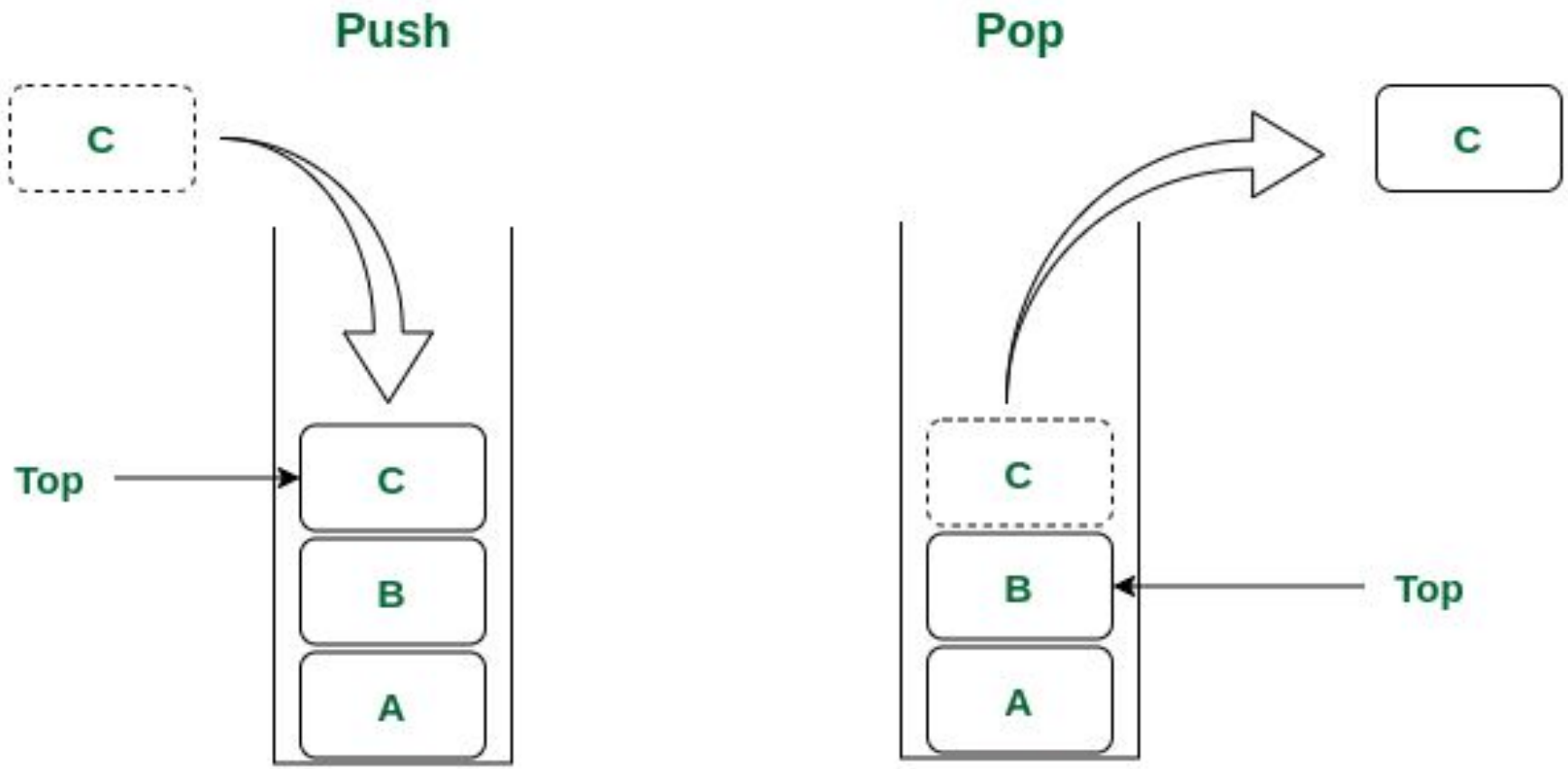
Stack

A **Stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle.

Stack operations:

- **push**: Add an element to the top of the stack.
- **pop**: Remove the element from the top of the stack.
- top: View the top element without removing it.
- empty: Check if the stack is empty.
- size: Get the number of elements in the stack.

Stack



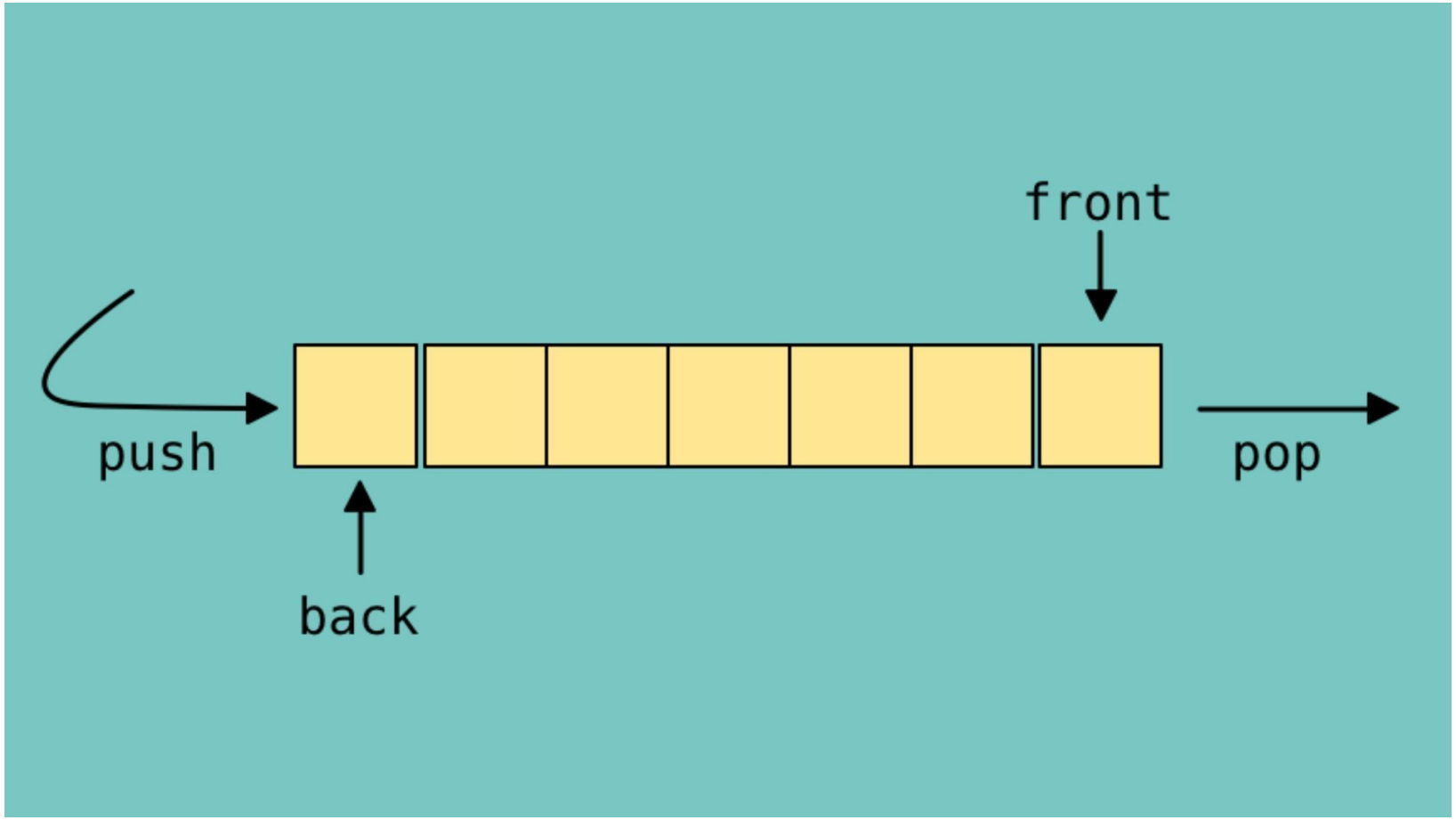
Queue

A **queue** is a linear data structure that follows the FIFO (First In, First Out) principle.

Queue operations:

- **push**: Add an element to the back of the queue.
- **pop**: Remove the element from the front of the queue.
- **front**: get the front element without removing it.
- **empty**: Check if the queue is empty.
- **size**: Get the number of elements in the queue.

Queue

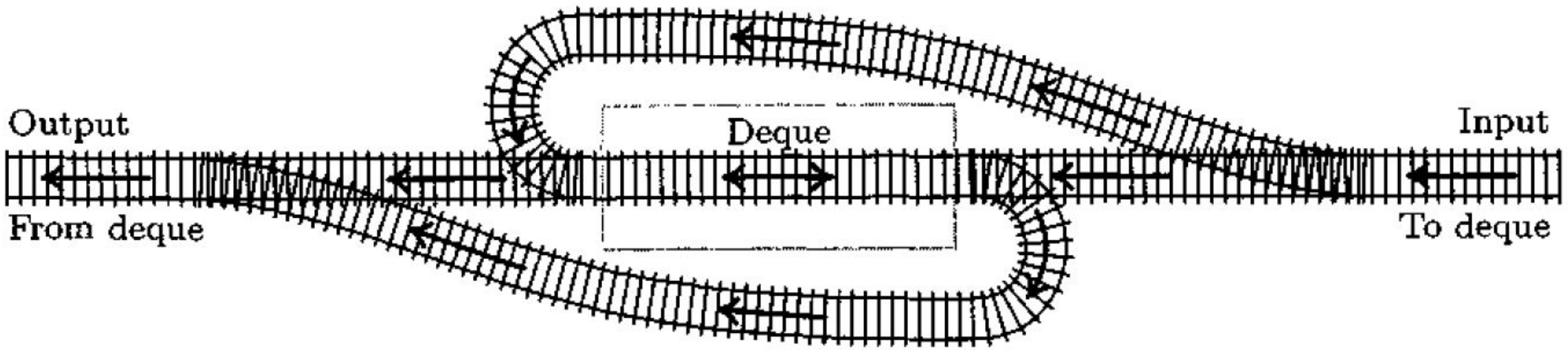


Deque

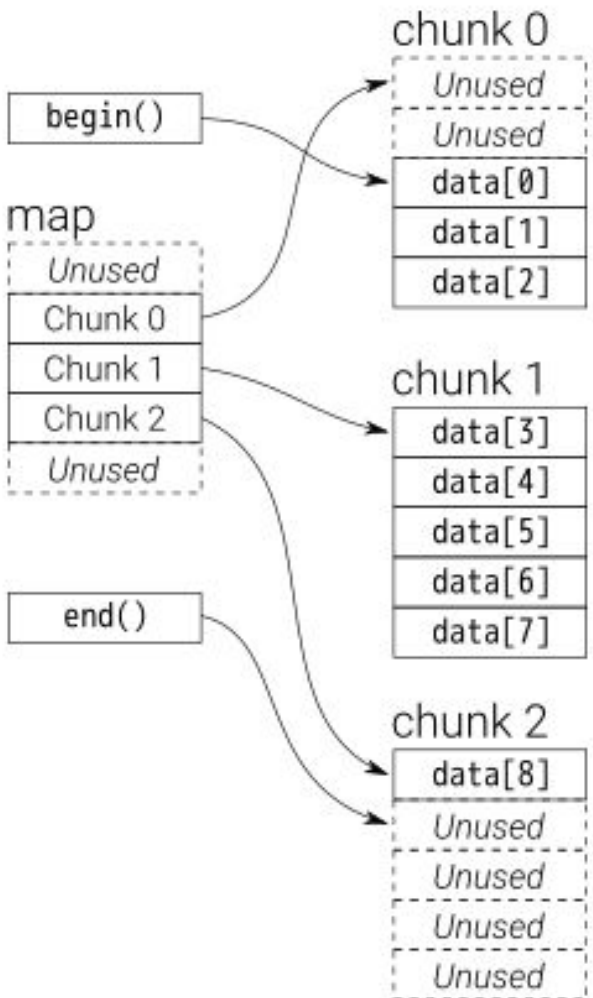
A **deque (double-ended queue)** is a linear data structure that allows insertion and deletion from both ends

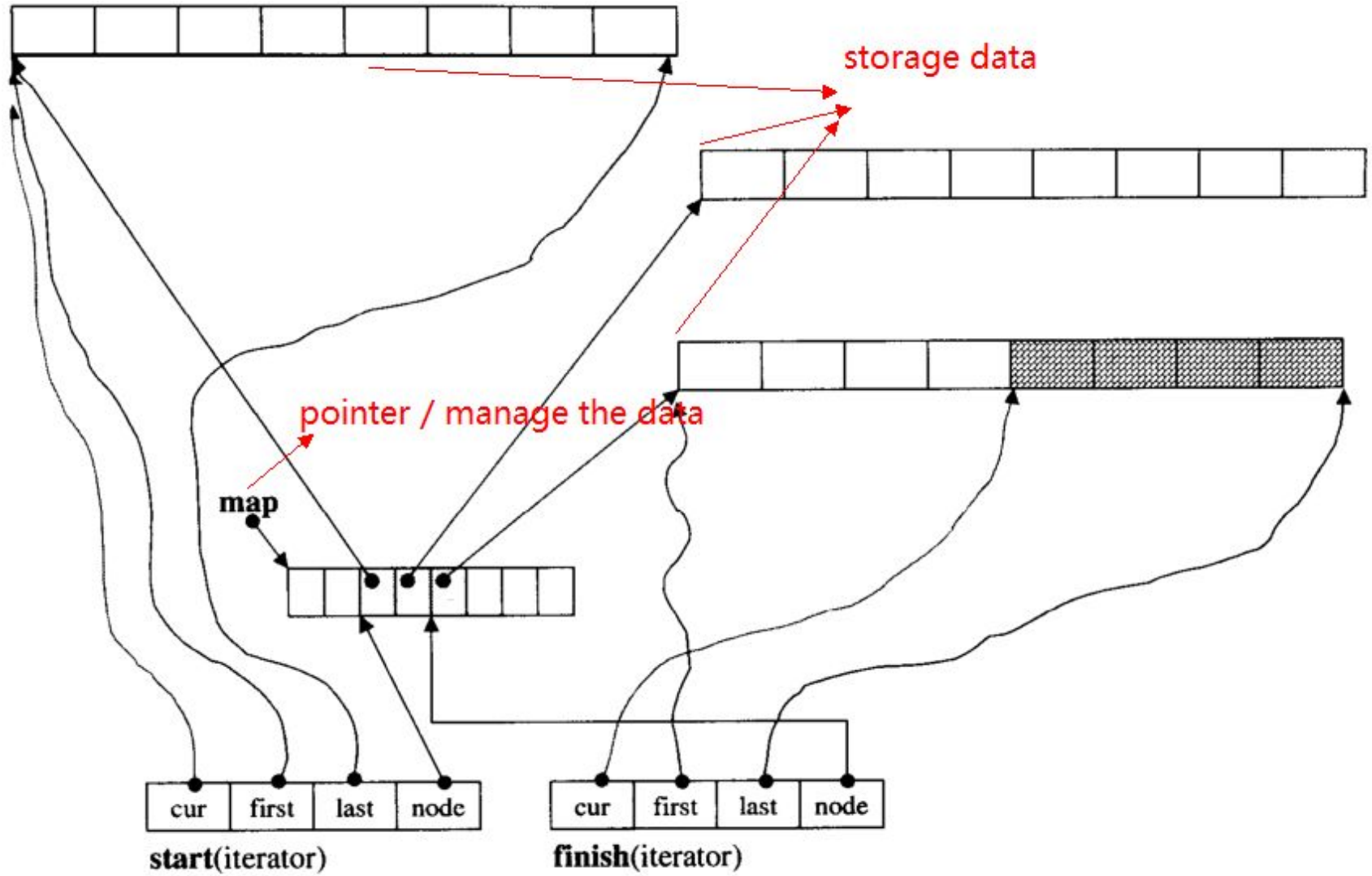
- **push_front**: Insert an element at the front.
- **push_back**: Insert an element at the back.
- **pop_front**: Remove an element from the front.
- **pop_back**: Remove an element from the rear.
- **front**: Get the front element without removing it.
- **back**: Get the rear element without removing it.
- **empty**: Check if the deque is empty.
- **size**: Get the number of elements in the deque.
- ***operator[]****: Get the element*

Deque



Deque


chunk 0
Unused
Unused
data[0]
data[1]
data[2]
begin()
map
Unused
Chunk 0
Chunk 1
Chunk 2
Unused
chunk 1
data[3]
data[4]
data[5]
data[6]
data[7]
end()
chunk 2
data[8]
Unused
Unused
Unused
Unused


storage data
pointer / manage the data
map
cur
first
last
node
start(iterator)
cur
first
last
node
finish(iterator)

Thank You